PACKETSCOPE

Web-Based PCAP and Network Traffic Analysis Platform
Defensive Cybersecurity Engineering System

A Capstone Project Report Submitted in Partial Fulfillment
of Requirements for the Degree of Bachelor of Technology in Computer Science

Candidate Name: Shivansh Dubey
Department of Computer Science & Engineering
Repository: https://github.com/ShivanshDubeyONG/PacketScope.git
Date of Submission: October 2026

1. Abstract and Keywords

Network packet analysis represents an indispensable pillar of modern defensive cybersecurity operations. However, legacy desktop packet analyzers such as Wireshark often present steep learning curves, lack integrated rule-based threat heuristics, and cannot be readily consumed within modern web architectures. This project presents **PacketScope**, a robust web-based network traffic analysis and forensic inspection platform designed to dissect .pcap and .pcapng capture files, aggregate communication telemetry, execute transparent deterministic threat heuristics, and export intelligence in open standard formats.

PacketScope genuinely implements four core modules from the prescribed curriculum: **PCAP to SQLite storage and export**, **PCAP to structured XML generation**, **PyShark dissector integration**, and **Wireshark/TShark protocol compatibility**. By leveraging FastAPI, SQLAlchemy, SQLite, React, TypeScript, and Tailwind CSS, PacketScope delivers a defensive operations dashboard that prioritizes explainability, deterministic alert triage, and zero credential exposure.

Keywords

Packet Capture (PCAP), Network Traffic Analysis, PyShark, TShark, Intrusion Detection Heuristics, PCAP to SQLite, PCAP to XML, Defensive Cybersecurity.

2. Introduction and Problem Statement

2.1 Operational Context

As enterprise networks and cloud environments grow increasingly heterogeneous, packet captures remain the definitive ground truth for diagnosing security incidents, protocol anomalies, and unauthorized reconnaissance. Despite the ubiquity of libpcap capture utilities, junior cybersecurity practitioners and security operations center (SOC) analysts frequently struggle to distill thousands of raw hexadecimal frames into actionable defensive intelligence.

2.2 Problem Statement

Existing network analysis tools suffer from several operational constraints: Desktop silos requiring heavy local installations, opaque threat scoring without verifiable evidence, vendor lock-in preventing easy database extraction, and credential exposure risks where unencrypted credentials in packet payloads are displayed indiscriminately. PacketScope addresses these challenges through a modern web platform.

3. Objectives and Prescribed Modules

3.1 Key Objectives

1. Develop an asynchronous backend capable of processing .pcap and .pcapng files up to 50 MB.

2. Extract standardized network telemetry across Ethernet, IPv4, IPv6, TCP, UDP, ICMP, DNS, and HTTP.

3. Compute genuine capture-level aggregates including throughput, packets-per-second, talker distributions, and time series.

4. Implement 8 deterministic, rule-based detection heuristics with structured forensic evidence.

3.2 Implemented Prescribed Modules

• **PCAP to SQLite**: Ingests and persists all packet metadata, alerts, and statistics to SQLite, with a downloadable standalone .sqlite database export service.

• **PCAP to XML**: Generates well-formed, structured XML containing capture metadata, statistical aggregates, detection alerts, and packet trees.

• **PyShark Engine**: Native Python packet parsing using PyShark wrapping TShark dissectors.

• **Wireshark / TShark Inspection**: High-performance protocol dissection compatible with standard Wireshark captures.

4. Background: PCAP, PyShark and TShark

The standard PCAP format, originated by the libpcap project, stores raw network frames preceded by a 24-byte global header and individual packet headers containing microsecond timestamps and length attributes. The newer PCAPNG format extends this with structured blocks, supporting multiple interfaces, comments, and nanosecond resolution.

Wireshark and its headless counterpart TShark represent the gold standard in protocol dissection, supporting over 3,000 protocol decoders. PyShark exposes TShark's dissection engine to Python via asynchronous subprocess communication. In PacketScope, PyShark is paired with an automated TShark locator and a Scapy fallback parser to provide bulletproof parsing resilience.

5. Requirements and Technology Stack

The PacketScope architecture is built using modern production-grade technologies:

• **Backend API**: Python 3.11, FastAPI (asynchronous ASGI framework), Uvicorn server.

• **Data Persistence**: SQLAlchemy 2.0 ORM with SQLite (foreign keys enforced via PRAGMA).

• **Packet Dissection**: PyShark with system TShark, supported by Scapy.

• **Frontend Architecture**: React 18, TypeScript, Vite build tool.

• **Styling System**: Vanilla Tailwind CSS with custom light-blue accent palette.

• **Data Visualization**: Recharts (responsive area charts and doughnut charts).

• **Containerization**: Docker and Docker Compose multi-stage containers.

6. System Architecture and Workflow

PacketScope adopts a decoupled monorepo architecture. The user uploads a capture file via the React client. The FastAPI backend validates the file extension (.pcap/.pcapng) and checks binary magic bytes (e.g. 0xa1b2c3d4 or 0x0a0d0d0a). The file is saved under a UUID-based filename. PyShark parses the packet stream, extracting headers and timestamps without saving raw payload content. Statistics and detection rules are evaluated, and all records are persisted in SQLite. The frontend polls or retrieves results via REST API.

7. Database Design and Processing Methodology

The database schema consists of four relational tables with cascading integrity:

1. **captures**: Core capture metadata, file hashes (SHA-256), duration, packet count, risk scores, and statuses.

2. **packets**: Indexed records of dissected packets including timestamps, MACs, IPs, ports, protocols, flags, and DNS/HTTP metadata.

3. **capture_statistics**: Aggregated metrics, protocol distribution JSON, port usage, top talkers, and time series.

4. **alerts**: Transparent detection records with rule IDs, severities, explanations, JSON evidence, and defensive recommendations.

8. Detection Rules and Risk Scoring

PacketScope implements 8 configurable, deterministic threat heuristics:

1. **RULE_PORT_SCAN (HIGH)**: >= 15 distinct ports contacted on a target within 10 seconds.

2. **RULE_CONNECTION_BURST (MEDIUM)**: >= 50 connection requests within 5 seconds.

3. **RULE_REPEATED_TCP_SYN (HIGH)**: >= 20 SYNs with >= 3:1 ratio over FIN/ACKs (SYN flood indicator).

4. **RULE_SUSPICIOUS_PORT_ACTIVITY (HIGH)**: Communication on monitored ports (21, 23, 69, 4444, 1337, etc.).

5. **RULE_ICMP_FLOOD (MEDIUM)**: >= 30 ICMP packets within 5 seconds.

6. **RULE_LARGE_DNS_QUERY (HIGH)**: DNS query names >= 60 characters (DNS tunneling indicator).

7. **RULE_UNENCRYPTED_PROTOCOL (LOW)**: Plaintext HTTP, FTP, or Telnet detected.

8. **RULE_TRAFFIC_CONCENTRATION (LOW)**: Single source host generating >= 60% of total capture traffic.

Composite Heuristic Risk Score: 0-100 calculated from weighted alert severities with transparent disclaimers.

9. Frontend Implementation & User Interface

The user interface follows a modern cybersecurity operations center design with an off-white background (#f8fafc), crisp white cards (#ffffff), and a light-blue accent palette (#38bdf8 / #0ea5e9). Key views include:

• **Network Traffic Overview**: Risk Score card, packet metrics, throughput volume, Traffic Over Time area chart, and Protocol Distribution doughnut chart.

• **Security Alerts & Anomalies**: Filterable alert cards with forensic evidence JSON drawers and defensive mitigation recommendations.

• **Packet Explorer**: Server-side search and filtering across IPs, ports, and protocols with deep packet inspection modals.

• **Data Exports Panel**: Downloadable SQLite database, structured XML, packet CSV, and JSON intelligence reports.

10. Testing, Results and Ethical Considerations

Testing was performed using a safe synthetic PCAP containing controlled traffic patterns generated via Scapy.

• **Automated Test Suite**: 22 Pytest backend tests covering validation, parsing, rules, statistics, exports, and missing-TShark handling.

• **Live Verification**: Successfully dissected 235 packets via PyShark, verified all 8 alerts, and confirmed XML/SQLite export integrity.

• **Ethical Scope**: PacketScope is strictly a defensive analysis platform. Raw payloads are excluded to ensure zero credential leakage.

11. Conclusion, Future Scope and References

PacketScope proves that network traffic analysis can be made intuitive, explainable, and accessible through web technologies. By implementing PCAP to SQLite, PCAP to XML, and PyShark integration, the system provides genuine forensic utility.

Future Scope

1. Live streaming interface capture via WebSockets.
2. Suricata / Snort rule format compatibility.
3. GeoIP map visualization for external endpoint traffic.

References

1. Postel, J. (1981). Transmission Control Protocol. RFC 793, IETF.
2. Mockapetris, P. (1987). Domain Names - Concepts and Facilities. RFC 1034, IETF.
3. Orebaugh, A., et al. (2006). Wireshark Protocol Analyzer Toolkit. Syngress.
4. Tiangolo, S. (2024). FastAPI: Modern, Fast Web Framework. tiangolo.com